1. Objeto

El presente pliego establece las condiciones técnicas mínimas y obligatorias que deberá cumplir la solución desarrollada en el marco del Reto 2 para su aceptación técnica.

El sistema deberá consistir en una solución software capaz de gestionar información cifrada, procesar ficheros, aplicar mecanismos de protección de datos y ofrecer una interfaz funcional para su utilización.

El incumplimiento de cualquiera de las condiciones establecidas podrá implicar la no aceptación técnica de la solución.

2. Alcance

Cada equipo deberá desarrollar una solución completa y funcional que integre las aportaciones de los distintos módulos implicados en el reto. La solución deberá incorporar, como mínimo:

- Aplicación ejecutable
- Gestión de mensajes cifrados
- Tratamiento de ficheros
- Integración funcional entre componentes
- Interfaz de usuario operativa
- Implementación de cifrado Vigenère
- Implementación de cifrado AES sobre ficheros
- Repositorio de código fuente
- Pipeline de CI/CD operativo
- Documentación técnica
- Demostración funcional

El objetivo es construir una solución técnicamente sólida, utilizable y defendible.

3. Principios generales

La solución deberá basarse en los siguientes principios:

- El sistema debe ejecutarse de forma autónoma.
- Las funcionalidades principales deben integrarse en una única solución.
- Los algoritmos solicitados deberán implementarse de forma funcional.
- El tratamiento de datos deberá ser correcto, repetible y justificable.
- La calidad del software deberá evidenciarse mediante automatización.
- El sistema deberá ser mantenible y defendible técnicamente.
- El equipo deberá poder justificar las decisiones adoptadas.

4. Condiciones técnicas de aceptación

4.1 Funcionamiento general

La aplicación deberá ejecutarse correctamente siguiendo la documentación entregada y permitir el uso real de sus funciones principales. No se aceptarán configuraciones ocultas ni dependencias no documentadas.

4.2 Integración de la solución

La entrega deberá presentarse como una única solución coherente. Las aportaciones de los distintos módulos deberán estar integradas y formar parte de un flujo funcional completo.

4.3 Vigenère en Python

La parte correspondiente a Vigenère deberá estar desarrollada en Python.

Deberá permitir cifrar y descifrar mensajes correctamente mediante una clave, trabajando con texto introducido por el usuario, leído desde fichero o mediante otro mecanismo equivalente definido por el equipo.

El comportamiento respecto a espacios, símbolos y caracteres especiales deberá estar documentado.

4.4 AES sobre ficheros

La solución deberá incorporar cifrado y descifrado de ficheros mediante AES.

El sistema deberá generar ficheros cifrados y permitir recuperar posteriormente el contenido original, gestionando de forma coherente la clave o contraseña utilizada.

4.5 Tratamiento de ficheros y recursos

La solución deberá gestionar la lectura y escritura de ficheros de texto y binarios, sus rutas y destinos, así como los errores de acceso y procesamiento. El cifrado y descifrado AES deberán permitir recuperar exactamente los bytes del fichero original, sin modificarlo ni sobrescribirlo indebidamente.

Para la aceptación se comprobará el comportamiento del proceso AES con ficheros de 0 bytes (resultado válido o rechazo controlado y justificado), 1 byte, 1 MiB y al menos 20 MiB. En el contrato técnico bastará con explicar brevemente cómo se prevé trabajar con ficheros de distintos tamaños e indicar las limitaciones conocidas. No se impondrá una técnica de implementación concreta; la capacidad real se comprobará con la solución final.

4.6 Interfaz de usuario

La solución deberá disponer de una interfaz funcional, ya sea gráfica, por consola, web o equivalente.

Desde dicha interfaz deberán poder utilizarse las funciones principales del sistema sin intervención técnica externa.

4.7 Control de versiones, trazabilidad y CI/CD

GitHub será el repositorio de trabajo y el medio habitual para compartir el código. No se utilizarán Drive, archivos ZIP, correo o mensajería para intercambiar versiones del proyecto como alternativa a Git. Cada integrante utilizará su propia identidad Git.

Durante las 3 semanas de desarrollo deberán quedar, como mínimo, estas evidencias:

- **Commits.** Al menos tres commits con cambios reales por integrante, realizados durante el desarrollo y con mensajes que describan brevemente qué se ha hecho.
- **Ramas.** Al menos dos ramas de desarrollo utilizadas por el equipo, además de la rama principal.
- **Pull Requests.** Al menos una PR integrada, con una descripción breve y una revisión o comentario de otro integrante.

En la evaluación se comprobarán el autor, las fechas y el contenido de los commits, la claridad de sus mensajes, el uso de ramas, la PR y la participación de cada integrante. No se valorará acumular commits o ramas sin sentido ni subir toda la solución de golpe al final. En programación en pareja podrán justificarse las aportaciones mediante coautoría y explicación del trabajo realizado.

Además, deberá existir un pipeline de CI/CD configurado en el repositorio que ejecute al menos una validación objetiva, como compilación, pruebas automáticas, análisis estático o equivalente, mostrando un resultado visible. Siempre que la plataforma lo permita, dicha validación deberá ejecutarse también sobre las Pull Requests antes de su integración. El uso de Git deberá ajustarse a la Guía de control de versiones y trabajo colaborativo con Git vigente para el curso.

Se recomiendo el uso de las siguientes herramientas:

- Github Actions para automatizar ejecuciones (test, cobertura de test, formato...)
- ACT (simulación de Github Actions en contenedor local)
- Pre-Commit Hook (control de commits en local): control de donde se hace push (bloqueo de ramas main y develop).

4.8 Cliente de interacción

El sistema deberá incluir un cliente (por ejemplo, aplicación de escritorio) que permita:

- seleccionar un fichero de entrada y, cuando proceda, la ubicación del fichero de salida
- cifrar y descifrar ficheros utilizando los mecanismos implementados, verificando que el contenido original pueda recuperarse
- que el código JavaFX de esa aplicación sea explicado con precisión públicamente
- La aplicación de escritorio será la misma para Linux que para Windows, solo se cambiará la exportación.
- El resultado será un .jar de tipo fatjar empaquetado con todos los elementos
- La entrada de archivos y su gestión informa al usuario mediante alertas y es capaz de controlar el flujo de datos mediante try, catch y finally

4.9 Documentación del proyecto

Al finalizar Discovery, cada equipo entregará **un contrato técnico con la propuesta inicial** de tratamiento de ficheros descrita en el Anexo VI**, junto con el anexo de planificación ágil, en un único PDF**. Se revisará para su aprobación técnica antes del desarrollo principal; los cambios sustanciales deberán quedar justificados. En la entrega final, además, se aportarán:

- Manual de instalación
- Manual de uso
- Decisiones técnicas: descripción breve del tratamiento de ficheros realizado, mecanismos utilizados y justificación de al menos una decisión relevante adoptada.
- Validación: evidencias de que el fichero original puede recuperarse correctamente tras el proceso de cifrado/descifrado, indicando posibles limitaciones o mejoras.

4.10 Aplicación móvil

La solución deberá incorporar una aplicación móvil nativa para Android adaptada al trabajo táctico sobre el terreno (rol de Auror desplegado):

- **Propósito y permisos del rol**: La aplicación operará exclusivamente en modo consulta/descifrado. No se requiere de la funcionalidad de cifrar mensajes nuevos, sino validar e interpretar la información recibida sobre el terreno.

- **Importación y recepción de recursos**: La app deberá permitir abrir o recibir archivos en formato texto plano (.txt) e imágenes (.png) procedentes de cualquier vía (almacenamiento local del dispositivo, envío mediante la función nativa de compartir, transferencia por red, ...)

- **Descifrado de imágenes mágicas (AES):** La aplicación dispondrá de controles de interfaz para que el usuario introduzca la clave AES correspondiente y descifre los archivos .png recibidos, permitiendo visualizar la imagen resultante directamente en la pantalla.

- **Descifrado de órdenes de batalla (César):** La aplicación permitirá indicar el desplazamiento del algoritmo César sobre los archivos .txt para recuperar el texto plano con las instrucciones de la contienda y mostrarlo en pantalla.

- **Gestión de errores y experiencia de usuario:** Ante la introducción de claves incorrectas, desplazamientos inválidos o archivos corruptos, la aplicación deberá capturar las excepciones y notificar al usuario mediante un aviso nativo (Toast, Snackbar, o dialogo emergente) sin cerrar inesperadamente la aplicación.

4.11 Aplicación móvil / Unity

La solución deberá incorporar una aplicación interactiva desarrollada en Unity 6.3 que funcione como cliente visual jugable del reto. Esta aplicación no sustituye a los requisitos técnicos de cifrado, ficheros o backend definidos en el pliego general, sino que deberá representar de forma autónoma la mecánica principal del reto mediante un minijuego 2D de descifrado.

La aplicación deberá incluir una escena principal donde un enemigo o personaje avance automáticamente hacia una puerta u objetivo a proteger. El jugador deberá visualizar un texto cifrado, introducir una respuesta mediante la interfaz y validar dicha respuesta desde la propia aplicación.

La lógica de juego deberá contemplar, como mínimo, los siguientes estados: partida en curso, victoria y derrota. Se valorará especialmente el uso correcto de componentes de física 2D, como Collider2D, Rigidbody2D, OnTriggerEnter2D u OnCollisionEnter2D, siempre que la configuración sea coherente y defendible.

4.12 Implementación de test unitarios

Se deberán implementar test unitarios para validar la aplicación. Estos test se deberán ejecutar en cada push dentro del ciclo de CI/CD, para asegurar que la calidad del código en el repositorio central es correcta.

La cobertura de test unitarios del proyecto debe de ser de al menos el 80%.

4.13 Estructura de proyecto y configuración local

El proyecto deberá tener una estructura de proyecto centralizada, fácil de replicar para los diferentes desarrolladores.

En el desarrollo local, se deben crear hooks de pre-commit para asegurar la calidad del código mediante test unitarios y se debe evitar el push directo a ramas comunes.

Se valora la alternativa de ejecuciones de pipelines CI/CD en local para no depender del repositorio en la nube para dichas ejecuciones.

5. Condición de no aceptación

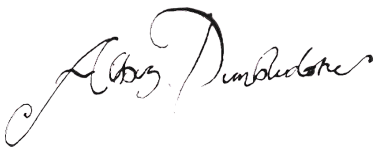
El incumplimiento de **cualquiera** de los puntos anteriores implicará la **no aceptación técnica del backend**, sin posibilidad de evaluación posterior ni compensación por funcionalidad del cliente o presentación visual.

6. Observación final

Este pliego no evalúa únicamente volumen de código o apariencia visual.

Evalúa especialmente funcionalidad real, integración técnica, correcta implementación de los mecanismos solicitados, calidad del software, automatización del proceso de validación y capacidad profesional de entrega.

Departamento de IT



Proveedor

Anexo I - ADAT

Req	Descripción	RA	Indicadores de logro
4.3 - 4.5	Gestión, transformación y cifrado de información almacenada en ficheros	RA1	I1.1, I1.2
4.6 - 4.8	Integración y validación funcional de la solución basada en ficheros	RA1	I1.1, I1.3
4.9	Documentación y justificación de la solución desarrollada	RA1	I1.2, I1.3

Anexo II - DEIN

Req	Descripción	RA	Indicadores de logro
4.8	Cliente de interacción	RA1,2,7	
4.9	Documentación del proyecto	RA6	

Anexo III - UNITY

Req	Descripción	RA	Indicadores de logro
4.11	Aplicación móvil / Unity	RA2, 3, 4, 5	I2.1, I3.2, I3.3, I4.1, I4.3, I4.5, I5.1, I5.3

Anexo IV - PYTH

Req	Descripción	RA	Indicadores de logro
4.12	Implementación de test unitarios coherentes y de calidad	RA2	
4.12	Cobertura de código (≥80%)	RA2	
4.7	Ciclo CI/CD en cada push	RA1	
4.13	Control de commits locales	RA1	
4.13	Configuración centralizada de proyecto	RA1	

Anexo V - PROM

Req	Descripción	RA	Indicadores de logro
4.10	Aplicación móvil nativa Android: Consulta y descifrado táctico (AES / César), tratamiento de recursos y gestión de errores	RA1, RA2, RA3	**I1.1, I1.2 I2.1, I2.2, I2.3 I3.1, I3.2, I3.3**

Anexo VI. Contenido mínimo del contrato técnico

La propuesta deberá explicar, de forma clara y breve, cómo se prevé gestionar la información almacenada en ficheros. Se solicita un diseño inicial que permita valorar su viabilidad, no una documentación completa de la aplicación antes de desarrollarla.

VI.1. Ficheros de la solución

El contrato técnico deberá identificar:

- **Entradas y salidas.** Qué tipos de ficheros se utilizarán, cuáles se generarán y qué función tendrá cada uno.
- **Almacenamiento.** Dónde se guardarán los resultados, qué ocurrirá con el fichero original y qué información deberá conservarse para poder recuperar el contenido.

VI.2. Funcionamiento propuesto

La propuesta incluirá una explicación sencilla del recorrido de la información:

- **Esquema general.** Un único diagrama de flujo o de componentes que represente el proceso desde la selección del fichero hasta la obtención del resultado. No se exigirá una notación UML concreta.
- **Operaciones principales.** Breve descripción de cómo se leerán, cifrarán, descifrarán y escribirán los ficheros, y cómo colaborarán los componentes cuando sea necesario.
- **Decisión técnica.** Justificación breve de una elección relevante del equipo, por ejemplo el formato de salida, la forma de procesar los archivos o el mecanismo de integración. No será obligatorio comparar formalmente varias alternativas.

VI.3. Límites e incidencias

Deberán contemplarse los aspectos básicos que puedan afectar al funcionamiento:

- **Tamaños y recursos.** Explicar cómo se prevé tratar ficheros pequeños y grandes e indicar las limitaciones que ya se conozcan.
- **Errores y protección.** Describir la respuesta prevista ante al menos dos situaciones de error relevantes y cómo se evitará perder o sobrescribir indebidamente el fichero original.

VI.4. Comprobación de la propuesta

El contrato recogerá un plan de comprobación sencillo, con las pruebas que se consideran necesarias y el resultado esperado. Será suficiente incluir:

- **Recuperación.** Una prueba de cifrado y descifrado que permita comprobar que se recupera el contenido original.
- **Tamaños.** Una prueba con un fichero pequeño y otra con uno de al menos 20 MiB, conforme al apartado 4.5. Su ejecución se acreditará con la entrega final.
- **Incidencias.** Una prueba que muestre qué ocurre cuando falla una operación sobre un fichero.

Las pruebas de aceptación restantes previstas en el pliego también se verificarán sobre la solución entregada. Durante Discovery bastará con describir cómo se plantea comprobar el funcionamiento, sin adjuntar todavía resultados ni métricas.

VI.5. Presentación y revisión

Podrán utilizarse tablas sencillas, esquemas propios y explicaciones breves. No se exigirá una matriz de trazabilidad, varios diagramas ni una especificación completa de endpoints o modelos de base de datos, salvo que resulte necesaria para comprender la solución propuesta.

Este contenido formará parte del mismo PDF del contrato técnico y de la planificación inicial. No requerirá una entrega independiente. La revisión de la propuesta concluirá con Aprobado o Requiere correcciones. Las observaciones recibidas deberán atenderse antes de la aprobación técnica y los cambios importantes posteriores deberán justificarse.